

MBA EM **ENGENHARIA**
DE SOFTWARE

**Domain Driven Design
(DDD) II**

Prof. Guilherme Lima

A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.

Proibida a reprodução, total ou parcial, sem autorização.

Lei nº 9610/98

Aula 2

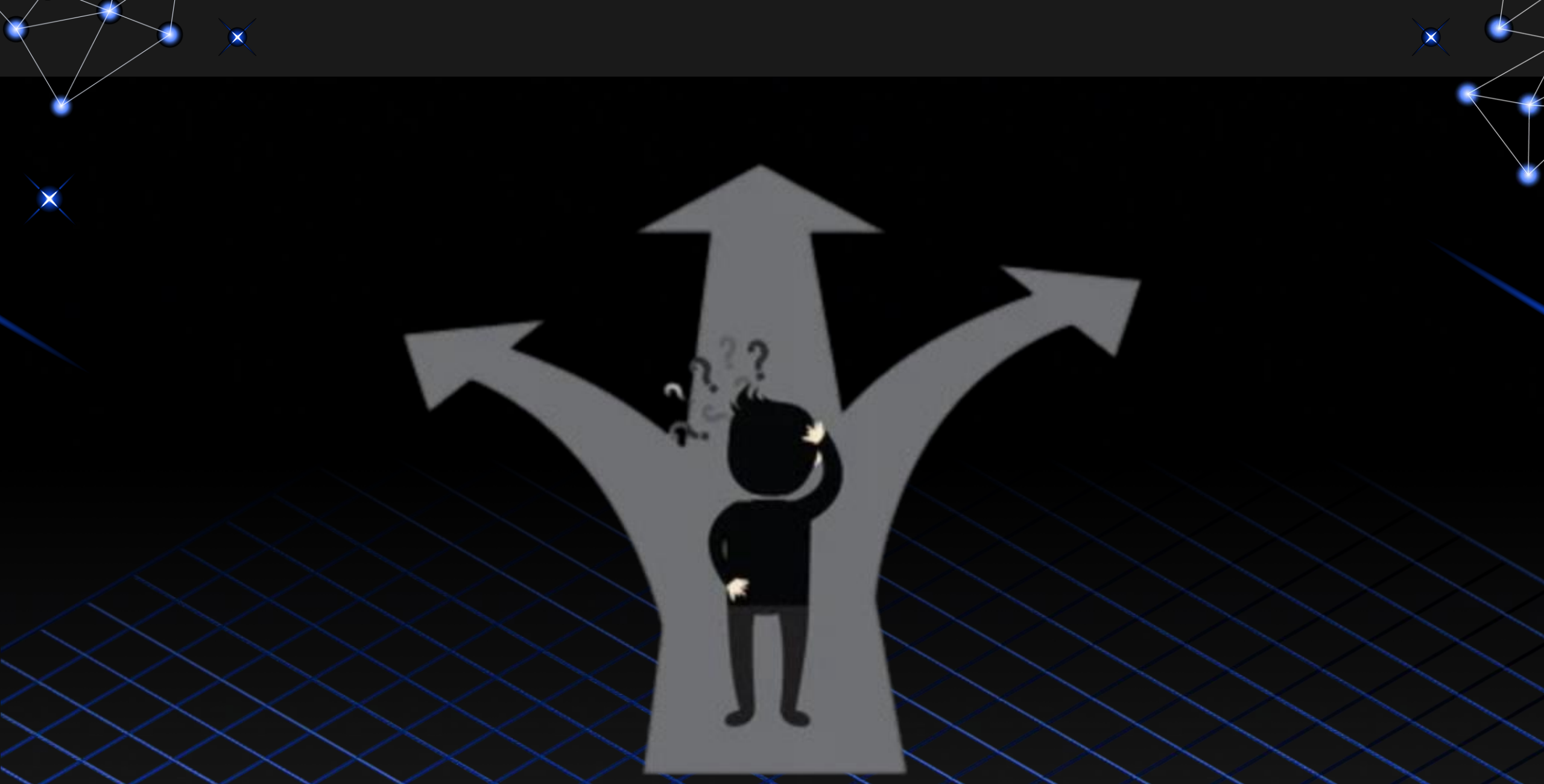
Domain-Driven Designer

Professor *Guilherme Lima*





DDD não se trata
exclusivamente
de **tecnologia**



Value Objects

DOMAIN-DRIVEN DESIGN / TACTICAL PATTERNS

Representar tudo com tipos primitivos

Os tipos primitivos representam apenas valores, sem comportamento ou validação, enquanto classes permitem:

- encapsular regras de negócio
- garantir consistência dos dados
- modelar entidades do domínio como objetos



E-mail e CEP inválidos entram sem resistência

Sem **Value Objects**, a validação é uma função solta que alguém precisa lembrar de chamar.

E-mail:

fulano.email.com ✓

CEP:

1234 ✓

DADOS INVÁLIDOS ACEITOS

A **confiança** no sistema é
perdida quando permitimos
que tipos primitivos
representem regras de
negócio.



Vamos analisar o projeto-1 sem DDD

A solução: cada tipo vira um Value Object

Em vez de tipos genéricos, cada conceito ganha sua própria classe. O tipo passa a carregar as **regras do negócio** dentro de si.

 Email CEP Endereco



Projeto 1

MODELO DE DOMÍNIO

DOMAIN-DRIVEN DESIGN / TACTICAL PATTERNS

AS 3 REGRAS DA BIBLIOTECA

01

Capacidade Máxima

O acervo não pode ultrapassar o limite de 5 livros simultâneos.

02

Disponibilidade

Um livro marcado como indisponível não pode ser emprestado.

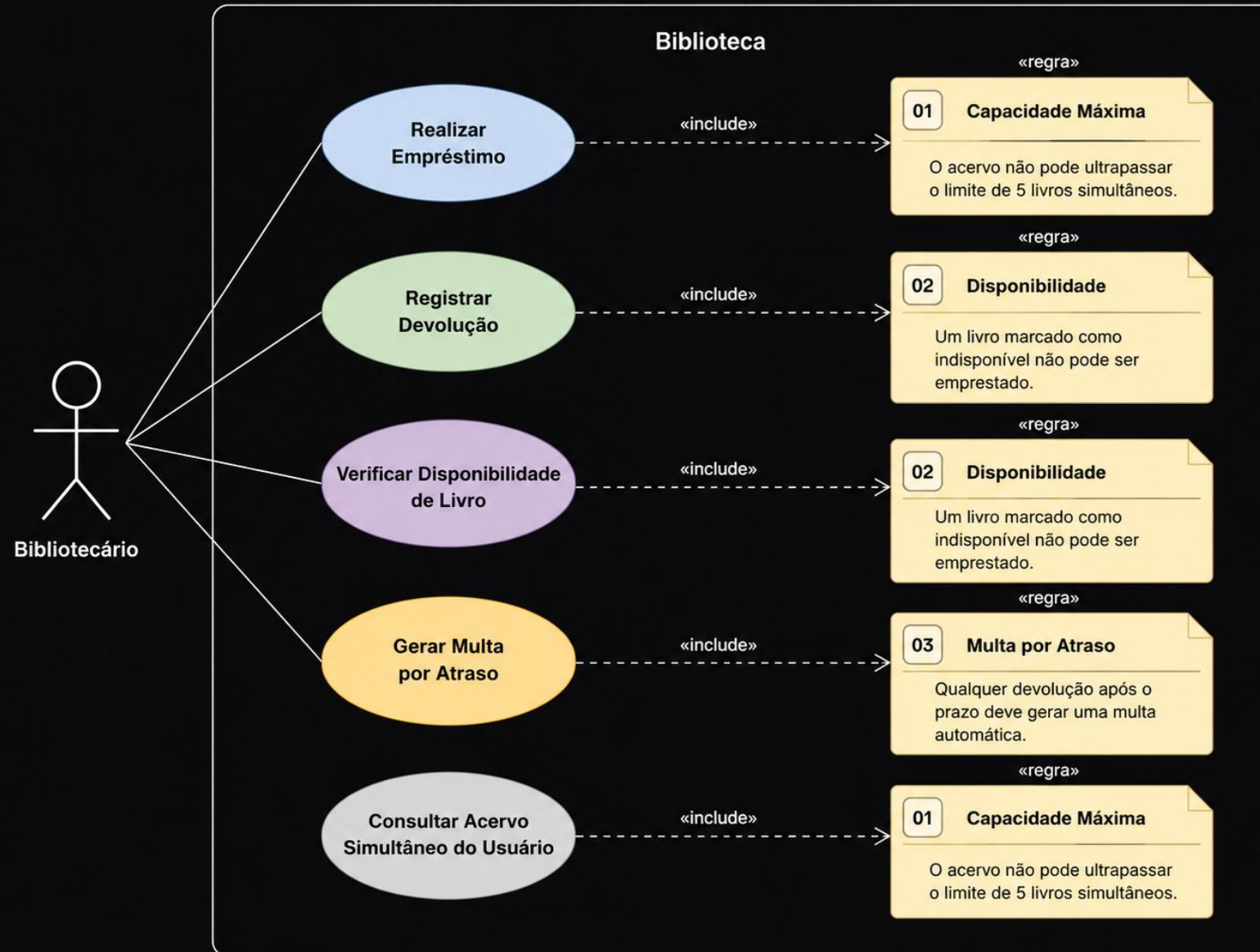
03

Multa por Atraso

Qualquer devolução após o prazo deve gerar uma multa automática.

Invariante: Uma regra de negócio que deve ser garantida em qualquer estado do sistema.

DIAGRAMA DE CASO DE USO – REGRAS (INVARIANTES) DA BIBLIOTECA



Vamos analisar o projeto-2 sem DDD

Problema 1 — O ACERVO CHEGA A 6 LIVROS

6

LIMITE: 5

Como a lista de livros é **pública**, qualquer parte do sistema pode adicionar itens diretamente.

```
biblioteca.livros.append(novo_livro);
```

A regra de negócio existe no papel, mas o código não oferece resistência à corrupção do estado.

Problema 2 — O MESMO LIVRO EMPRESTADO DUAS VEZES



ESTADO INVÁLIDO

Inconsistência de dados no banco

Problema 3 — A MULTA QUE NINGUÉM GARANTE APLICAR

”multa = conta solta, fácil de esquecer”

A SOLUÇÃO: BIBLIOTECA COMO RAIZ DE AGREGADO



AGGREGATE ROOT

A FRONTEIRA DE CONSISTÊNCIA: DE FORA SÓ SE LÊ



LEITURA

O estado é **público e livre**
para consulta por qualquer parte
do sistema.



ESCRITA

Alterações são **restritas** aos
métodos públicos da Raiz do
Agregado.

CADA COMANDO PROTEGE UMA INVARIANTE



`adicionar_livro()`

Verifica o tamanho atual do acervo e recusa a operação se o limite de 5 for atingido.



`emprestar()`

Valida o status de disponibilidade do livro antes de permitir a transição de estado.



`devolver()`

Calcula e aplica a multa automaticamente com base na data de devolução e prazo.



Projeto 3

Camada Anticorrupção (Anti-Corruption Layer)

DOMAIN-DRIVEN DESIGN / TACTICAL PATTERNS

SISTEMA LEGADO — NÃO ALTERE!

Quase todo sistema precisa conversar

com algo que **não controlamos**:

- a API de um banco
- uma biblioteca antiga
- um sistema de terceiros



Vamos analisar o projeto-3

Obrigado!

Prof. Guilherme Lima | <https://www.linkedin.com/in/guilherme-lima-developer>

MBA **USP**
ESALQ